

Национальный исследовательский университет "Высшая школа экономики"
Факультет компьютерных наук
Образовательная программа "Программная инженерия"

СОГЛАСОВАНО

Старший разработчик в X5 Tech

_____ В. Е. Козловский

« ____ » _____ 2026 г.

УТВЕРЖДЕНО

Академический руководитель
образовательной программы
"Программная инженерия", старший
преподаватель департамента
программной инженерии

_____ Н. А. Павлочев

« ____ » _____ 2026 г.

**РАЗРАБОТКА ИНТЕЛЛЕКТУАЛЬНОГО АССИСТЕНТА ДЛЯ
ОБРАЗОВАТЕЛЬНЫХ УЧРЕЖДЕНИЙ**

Программа и методика испытаний

ЛИСТ УТВЕРЖДЕНИЯ

RU.17701729.05.09 51 01-1-ЛУ

Исполнители:

Студент группы БПИ236

_____ / С. Е. Растворов /

« ____ » _____ 2026 г.

Подп. и дата	
Инв.№ дубл.	
Взам. инв.№	
Подп. и дата	
Инв.№ подл.	

УТВЕРЖДЕН

RU.17701729.05.09 51 01-1-ЛУ

**РАЗРАБОТКА ИНТЕЛЛЕКТУАЛЬНОГО АССИСТЕНТА ДЛЯ
ОБРАЗОВАТЕЛЬНЫХ УЧРЕЖДЕНИЙ**

Программа и методика испытаний

RU.17701729.05.09 51 01-1

Листов 14

Инов.№ подп	Подп. и дата	Взам. инв.№	Инов.№ дубл.	Подп. и дата

АННОТАЦИЯ

Программа и методика испытаний определяет объект, цель, средства, порядок и методы испытаний программного продукта «Разработка интеллектуального ассистента для образовательных учреждений».

Документ предназначен для проверки соответствия программы требованиям технического задания и фиксации критериев приемки результата разработки.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.09 51 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

СОДЕРЖАНИЕ

1. ОБЪЕКТ ИСПЫТАНИЙ	4
1.1. Наименование программы	4
1.2. Краткая характеристика области применения программы	4
2. ЦЕЛЬ ИСПЫТАНИЙ	5
3. ТРЕБОВАНИЯ К ПРОГРАММЕ	6
3.1. Требования к функциональным характеристикам	6
3.1.1. Состав выполняемых функций	6
3.1.2. Требования к входным данным	6
3.1.3. Требования к выходным данным	7
3.2. Требования к интерфейсу	7
4. ТРЕБОВАНИЯ К ПРОГРАММНОЙ ДОКУМЕНТАЦИИ	8
5. СРЕДСТВА И ПОРЯДОК ИСПЫТАНИЙ	9
5.1. Технические средства	9
5.2. Программные средства	9
5.3. Порядок проведения испытаний	9
5.4. Требования к персоналу	10
6. МЕТОДЫ ИСПЫТАНИЙ	11
6.1. Проверка требований к технической документации	11
6.2. Проверка требований к интерфейсу	11
6.3. Проверка требований к функциональным характеристикам	11
6.3.1. Качество RAG-ответа	11
6.3.2. Деграция до лексического fallback	11
6.3.3. Загрузка и чанкинг документов	11
6.3.4. История диалогов	12
6.3.5. Метрики	12
6.3.6. Автоматизированные тесты	12
СПИСОК ИСПОЛЪЗУЕМОЙ ЛИТЕРАТУРЫ	13

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.09 51 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

1. ОБЪЕКТ ИСПЫТАНИЙ

1.1. Наименование программы

Наименование программы: «Разработка интеллектуального ассистента для образовательных учреждений».

Наименование программы на английском языке: «Development of an Intelligent Assistant for Educational Institutions».

1.2. Краткая характеристика области применения программы

Программа предназначена для интеллектуальной поддержки студентов и преподавателей НИУ ВШЭ и других вузов — ответы на вопросы об учебных планах, регламентах, сроках и процедурах на основе внутренних документов. Система базируется на RAG-архитектуре и использует локальную векторную базу (ChromaDB), языковую модель (Ollama/Mistral), эмбединги (ruBERT-tiny2) и интерфейсы Telegram/Web. Разработка выполнена командой трёх студентов НИУ ВШЭ в рамках курсового проекта по направлению 09.03.04 «Программная инженерия».

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.09 51 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

2. ЦЕЛЬ ИСПЫТАНИЙ

Целью испытаний является проверка корректности выполнения программой функций, изложенных в разделе «Требования к программе» документа «Техническое задание», применительно к индивидуальной части разработки — RAG-ядру системы «Разработка интеллектуального ассистента для образовательных учреждений».

Испытания направлены на подтверждение следующих функций, реализованных в зоне ответственности разработчика: семантический поиск релевантных фрагментов в ChromaDB по запросу пользователя с использованием эмбеддингов ruBERT-tiny2; генерация ответа с помощью LLM (Ollama/Mistral) на основе найденного контекста с указанием источников; деградация до лексического резервного поиска при недоступности LLM; загрузка, нормализация и чанкинг документов форматов PDF, HTML, TXT и изображений с OCR; фоновая индексация корпуса в ChromaDB без прерывания сервиса; хранение истории диалогов пользователя в PostgreSQL; сбор метрик качества; аутентификация запросов по токену с ролевой моделью; обработка запросов через эндпоинты POST /ask, GET /health, POST /index, GET /metrics.

Границы приёмки совпадают с реализованными функциями, подтверждёнными исходным кодом в src/server/app/ и тестовыми модулями в tests/. Функции, заявленные в техническом задании, но не реализованные в коде, исключаются из приёмочного контроля.

Результаты испытаний считаются положительными при соблюдении следующих условий: не менее 85% вопросов из контрольного набора на основе индексированных документов получают корректный содержательный ответ; среднее время ответа POST /ask не превышает 7 секунд при доступном Ollama; лексический fallback активируется и возвращает ответ без остановки сервиса; все тесты в tests/ проходят без падений; сборка docker compose up завершается успешно.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.09 51 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

3. ТРЕБОВАНИЯ К ПРОГРАММЕ

3.1. Требования к функциональным характеристикам

3.1.1. Состав выполняемых функций

В рамках индивидуальной части разработки программа «Разработка интеллектуального ассистента для образовательных учреждений» должна выполнять следующие функции RAG-ядра и серверной части, установленные техническим заданием и проверяемые в ходе испытаний.

1. Приём вопроса пользователя через POST /ask с аутентификационным токеном; возврат ответа в формате JSON с текстом, перечнем источников и оценкой уверенности.
2. Семантический поиск: вычисление эмбединга запроса моделью ruBERT-tiny2; поиск k ближайших фрагментов по косинусному сходству в векторной базе ChromaDB.
3. Генерация ответа: формирование промпта с контекстом из найденных фрагментов и передача в LLM (Ollama/Mistral) через LangChain; включение цитат и ссылок на источники в ответ.
4. Лексический резервный поиск (src/server/app/lexical.py): активация при недоступности Ollama; ключевое совпадение без нейросетевой генерации; возврат HTTP 200 с ответом, помеченным как fallback.
5. Загрузка документов (src/server/app/document_ingestion.py): поддержка PDF, HTML, TXT; OCR изображений PNG/JPG через Tesseract; нормализация и разбивка на чанки до 500 символов с перекрытием.
6. Фоновая индексация (src/server/app/index_documents.py): запуск через POST /index; сохранение чанков и метаданных в ChromaDB; не прерывает обработку /ask.
7. История диалогов (src/server/app/conversation_memory.py): сохранение вопросов и ответов пользователя в PostgreSQL; выборка истории при построении контекста следующего запроса.
8. Метрики (src/server/app/metrics.py): измерение времени обработки каждого запроса; подсчёт confidence score; учёт доли эскалированных запросов; возврат через GET /metrics.
9. Аутентификация (src/server/app/auth_*.py): проверка Bearer-токена; разграничение прав по ролям «пользователь» и «администратор».
10. Healthcheck (GET /health): возврат HTTP 200 при корректном состоянии сервиса.

3.1.2. Требования к входным данным

Входными данными серверной части являются:

1. HTTP-запросы POST /ask в формате JSON: поле question (строка), опциональные поля user_id и параметры сессии; заголовок Authorization с Bearer-токеном.
2. HTTP-запросы POST /index: путь к каталогу или список файлов для индексации.
3. Документы PDF, HTML, TXT (до 20 МБ) и изображения PNG/JPG для загрузки в базу знаний.
4. Переменные окружения конфигурации (src/server/app/config.py): BOT_TOKEN, DATABASE_URL, LLM_HOST, CHROMA_PATH, ALLOW_LLM_FALLBACK.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.09 51 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

3.1.3. Требования к выходным данным

Выходными данными серверной части являются:

1. JSON-ответ POST /ask: поля answer (строка), sources (массив объектов с полями title, excerpt, page/link), confidence (число 0—1), fallback (булев флаг).
2. JSON-ответ GET /metrics: avg_response_time (секунды), mean_confidence, escalation_rate (доля).
3. HTTP-ответ GET /health: статус 200 и тело {«status»: «ok»}.

3.2. Требования к интерфейсу

REST API сервера реализован средствами FastAPI и предоставляет следующие маршруты:

1. POST /ask — основной диалоговый эндпоинт; принимает вопрос, возвращает ответ с источниками.
2. GET /health — healthcheck; возвращает HTTP 200 при работоспособном состоянии сервиса.
3. POST /index — запуск фоновой индексации корпуса документов; возвращает HTTP 202.
4. GET /metrics — возврат текущих метрик качества в JSON.

Доступ к маршрутам ограничивается проверкой Bearer-токена. Документация API доступна через /docs (Swagger UI). При отсутствии токена возвращается HTTP 401; при недостаточных правах — HTTP 403.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.09 51 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

4. ТРЕБОВАНИЯ К ПРОГРАММНОЙ ДОКУМЕНТАЦИИ

На испытание должна быть представлена документация в следующем составе:

1. «Разработка интеллектуального ассистента для образовательных учреждений». Техническое задание (ГОСТ 19.201-78 [7]).
2. «Разработка интеллектуального ассистента для образовательных учреждений». Пояснительная записка (ГОСТ 19.404-79 [10]).
3. «Разработка интеллектуального ассистента для образовательных учреждений». Программа и методика испытаний (ГОСТ 19.301-79 [8]).
4. «Разработка интеллектуального ассистента для образовательных учреждений». Текст программы (ГОСТ 19.401-78 [9]).
5. «Разработка интеллектуального ассистента для образовательных учреждений». Руководство оператора (ГОСТ 19.505-79 [11]).

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.09 51 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

5. СРЕДСТВА И ПОРЯДОК ИСПЫТАНИЙ

5.1. Технические средства

Испытания проводятся на рабочей станции, способной запустить контейнерный стенд проекта. Для работы RAG-пайплайна требуется доступный экземпляр Ollama с моделью Mistral и не менее 16 ГБ оперативной памяти. При использовании GPU для ускорения генерации — NVIDIA-карта с 8+ ГБ VRAM. Для тестирования лексического fallback доступность Ollama намеренно отключается.

В состав стенда входят: FastAPI RAG-сервер (src/server/), ChromaDB с персистентным индексом на диске, PostgreSQL 16 (история диалогов, аутентификация), Docker Compose для оркестрации. Для ручной проверки API используется HTTP-клиент (curl, httpie или Postman).

5.2. Программные средства

Для подготовки и проверки окружения используются следующие команды:

```
uv sync --all-packages --group dev
./uv-linters.sh
pytest tests/test_ask.py tests/test_rag.py tests/test_document_ingestion.py \
    tests/test_health.py tests/test_server_config.py
docker compose config
docker compose up -d
```

Тестовое покрытие включает: tests/test_ask.py — интеграционные тесты POST /ask; tests/test_rag.py — юнит-тесты retrieval и generation; tests/test_document_ingestion.py — тесты чанкинга и загрузки; tests/test_health.py — smoke-тест GET /health; tests/test_server_config.py — тесты конфигурации. Линтинг выполняется через ruff check и ruff format --check.

5.3. Порядок проведения испытаний

Испытания выполняются в следующем порядке.

1. Подготовка окружения: `uv sync --all-packages --group dev`; настройка `.env` с параметрами `DATABASE_URL`, `LLM_HOST`, `CHROMA_PATH`; `docker compose config` для валидации конфигурации.
2. Автоматизированные проверки: `./uv-linters.sh` (должна завершиться с кодом 0); `pytest tests/` (все тесты должны пройти без падений).
3. Запуск контейнерного стенда: `docker compose up -d`; проверка GET /health (ожидаемый ответ HTTP 200, {«status»: «ok»}).
4. Подготовка индекса: POST /index с тестовым набором документов; ожидание завершения фоновой индексации; проверка того, что /ask продолжает работать в период индексации.
5. Проверка RAG-ответа: направить тестовые вопросы через POST /ask с действующим токеном; зафиксировать ответы, источники, время обработки и значения confidence.
6. Проверка лексического fallback: остановить Ollama или установить `ALLOW_LLM_FALLBACK=1`; повторить POST /ask; убедиться, что ответ возвращается с HTTP 200 и флагом fallback без ошибки 500.
7. Проверка метрик: GET /metrics; убедиться в наличии числовых значений avg_response_time, mean_confidence, escalation_rate.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.09 51 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

8. Проверка аутентификации: запрос к POST /ask без токена должен вернуть HTTP 401; запрос с токеном пользователя к административному маршруту должен вернуть HTTP 403.
9. Фиксация результатов и сопоставление с критериями приёмки технического задания.

5.4. Требования к персоналу

Испытания должен проводить разработчик или инженер по тестированию, умеющий запускать Python-проект через Docker Compose, выполнять HTTP-запросы, читать журналы контейнеров и интерпретировать JSON-ответы API. Знание базовых принципов RAG-пайплайна приветствуется для корректной оценки качества ответов. Оценка доли корректных ответов (не менее 85%) выполняется экспертным сопоставлением ответов с содержанием индексируемых документов.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.09 51 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

6. МЕТОДЫ ИСПЫТАНИЙ

6.1. Проверка требований к технической документации

Проверка технической документации выполняется сверкой комплекта документов с перечнем ГОСТ 19.301-79. Дополнительно сопоставляются описания функций, интерфейсов и ограничений с содержимым модулей `src/server/app/`: `rag.py`, `vector.py`, `document_ingestion.py`, `index_documents.py`, `lexical.py`, `conversation_memory.py`, `metrics.py`, `auth_*.py`, `config.py`. Результат положителен, если в документации нет формулировок, противоречащих реализации, и все функции подтверждены кодом или тестами.

6.2. Проверка требований к интерфейсу

Проверка REST API выполняется последовательными HTTP-запросами с контролем: корректный JSON-ответ со структурой `answer/sources/confidence` при `POST /ask`; HTTP 200 при `GET /health`; корректные числовые поля при `GET /metrics`; HTTP 401 при отсутствии токена; HTTP 202 или 200 при `POST /index`. Для каждого маршрута фиксируется время ответа.

6.3. Проверка требований к функциональным характеристикам

Функциональные характеристики проверяются по сценариям, каждый из которых содержит фиксированные входные данные, последовательность действий и критерий успешности.

6.3.1. Качество RAG-ответа

Исходные данные: проиндексированный корпус документов (не менее 10 документов вуза) и контрольный набор из не менее 20 вопросов с известными ответами по содержанию документов. Действия: направить каждый вопрос через `POST /ask`; зафиксировать текст ответа, список источников, `confidence` и время обработки. Ожидаемый результат: ответ содержит информацию из документов с явным указанием источника; среднее время не превышает 7 секунд; значение `confidence` числовое и находится в диапазоне `[0, 1]`. Критерий успешности: не менее 85% вопросов оценены как корректные ответы при сопоставлении с документами.

6.3.2. Деградация до лексического fallback

Исходные данные: остановленный или недоступный Ollama (или `ALLOW_LLM_FALLBACK=1` в переменных окружения). Действия: `POST /ask` с произвольным вопросом. Ожидаемый результат: HTTP 200, ответ содержит текст, поле `fallback` равно `true`, сервис не завершается аварийно. Критерий успешности: ни один из трёх повторных запросов не вернул HTTP 5xx.

6.3.3. Загрузка и чанкинг документов

Исходные данные: набор тестовых документов — один PDF-файл, один HTML-файл, один TXT-файл, одно изображение с текстом (PNG). Действия: вызвать `document_ingestion` через `pytest` или `POST /index`; проверить созданные чанки в ChromaDB. Ожидаемый результат: PDF разбит на фрагменты не длиннее 500 символов с перекрытием; HTML — текст извлечён без HTML-тегов; изображение — текст извлечён через OCR. Критерий успешности: тест `tests/test_document_ingestion.py` проходит без ошибок.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.09 51 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

6.3.4. История диалогов

Исходные данные: два последовательных запроса POST /ask от одного пользователя. Действия: выполнить первый запрос, затем второй, ссылающийся на контекст первого; проверить таблицу диалогов в PostgreSQL. Ожидаемый результат: оба запроса и оба ответа сохранены в базе с корректным user_id и временной меткой. Критерий успешности: записи присутствуют в базе и содержат ожидаемые поля.

6.3.5. Метрики

Исходные данные: не менее 5 выполненных запросов POST /ask. Действия: GET /metrics. Ожидаемый результат: avg_response_time — число > 0 , mean_confidence — число в $[0, 1]$, escalation_rate — число в $[0, 1]$. Критерий успешности: все поля присутствуют и имеют числовой тип.

6.3.6. Автоматизированные тесты

Исходные данные: текущая ревизия репозитория. Действия: uv sync --all-packages --group dev; ./uv-linters.sh; pytest tests/. Ожидаемый результат: код выхода 0 для каждой команды; ни один тест не помечен как FAILED или ERROR. Критерий успешности: выход с кодом 0 и отсутствие падений.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.09 51 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ

1. ГОСТ 19.101-77: Виды программ и программных документов. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
2. ГОСТ 19.102-77: Стадии разработки. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
3. ГОСТ 19.103-77: Обозначения программ и программных документов. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
4. ГОСТ 19.104-78: Основные надписи. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
5. ГОСТ 19.105-78: Общие требования к программным документам. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
6. ГОСТ 19.106-78: Требования к программным документам, выполненным печатным способом. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
7. ГОСТ 19.201-78: Техническое задание. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
8. ГОСТ 19.301-79: Программа и методика испытаний. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
9. ГОСТ 19.401-78: Текст программы. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
10. ГОСТ 19.404-79: Пояснительная записка. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
11. ГОСТ 19.505-79: Руководство оператора. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
12. ГОСТ 19.603-78: Общие правила внесения изменений. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
13. ГОСТ 19.604-78: Правила внесения изменений в программные документы, выполненные печатным способом. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.09 51 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

ЛИСТ РЕГИСТРАЦИИ ИЗМЕНЕНИЙ

[illegible]